HAIRORIGINALS · PARTNER LEAD APP

HO Connect
Final Deployment Plan

How to put the API, the web dashboard and the Android app live for the 15–25 partner pilot. Everything runs on free tiers and stays on the existing Supabase project.

VERSION	DATE	REPLACES
1.0 (final)	7 November 2026	The earlier draft plan and LAUNCH.md §2–3

Decisions at a glance

Part	Where it runs	Notes
Database, Auth, Storage, SMS hook	Existing Supabase project **unchanged**	One project for testing and production. No new project, no data migration.
Backend API (`apps/api`)	Render free web service (Docker) + a free keep-alive ping	Exactly one instance. Move to Render Starter (~$7/mo) once real money is paid out.
Web dashboard + customer QR pages (`apps/dashboard`)	Vercel	Serves `/login` , `/r/<code>` , `/privacy` , `/delete-account` .
Android app (`apps/mobile`)	Expo EAS builds → Google Play internal testing	$25 one-time Play Console fee. No iOS and no web version for the pilot.
Partner login SMS	Supabase `send-sms` Edge Function → Nimbus	Already set up on the existing project; only verify it.

> **The one rule that comes with one shared Supabase project**
>
> The database your laptop uses *is* the production database. Read Section 2 before onboarding the first real partner: it covers the encryption key, the one-time clean-up, backups, and what you must stop doing locally once partners are live.

Checked against the repository on 7 Nov 2026 (Dockerfiles, `.env.example` , API env and jobs, `eas.json` , README, LAUNCH.md). Free-tier limits and prices change: confirm them when you sign up.

1 What gets deployed

Play Store (EAS build)

Partner app

Expo SDK 58, React Native. Registration, leads, QR codes, commissions. Android only for the pilot.

Vercel

Dashboard + customer pages

Next.js 16. Staff dashboard, customer QR / consent page, privacy and account-deletion pages.

Render

API

Node.js 22 + Fastify + Drizzle. REST API, customer consent SMS, push, background jobs every 10 min. Port from `$PORT`.

↓ app and dashboard call the API over HTTPS; the API talks to Supabase with the secret key ↓

Existing Supabase project (Mumbai), the same one used for testing

Postgres
Schema from Drizzle migrations 0000–0004

Auth
Partner phone OTP, staff email login

Storage
Private `partner-documents` bucket

Edge Function
`send-sms` hook → Nimbus (DLT, `HAIROG`)

External service	Used for
Nimbus SMS	Partner login codes (via the Supabase hook) and customer consent codes (via the API when `OTP_PROVIDER=nimbus`).
Expo Push	Partner notifications. Android delivery needs Firebase (FCM) credentials uploaded to EAS.
AI try-on app	Partner try-on QR codes open `https://aitryon.hairoriginals.com` with `utm_source=ho_connect`. That app must have the attribution change deployed.

> **Why the API must be a single instance**
>
> The API runs two jobs every 10 minutes inside its own process: it expires leads whose customer never confirmed, and it releases commissions whose hold period has ended. Never scale the Render service above one instance. The jobs only change rows that are still due, so a second copy would not corrupt data, but it could send duplicate notifications.

Corrections to the previous draft plan

Previous draft said	This plan
API uses Hono	The API uses **Fastify**. Nothing changes for deployment.
Next.js 15, Expo SDK 53	The code uses **Next.js 16** and **Expo SDK 58**.
Create a new production Supabase project	**Removed.** The existing project serves both testing and production.
Generate a new `DATA_ENCRYPTION_KEY` for production	**Reversed.** Use the same key as your current `.env` (Section 2.1).
Fly.io free tier, region `maa`	Fly.io no longer offers a free tier to new accounts, and `maa` is Chennai, not Mumbai. Fly.io is not used.
Expo web build of the app	**Dropped.** The app targets iOS and Android only, and the dashboard already serves every web page customers need.
Custom Vercel build command	Not needed. Vercel installs the npm workspaces from the repository root.

2 Running test and production on one Supabase project

Keeping one project saves cost and setup work. The trade-off is that every script, migration and local API you run touches real partner data. These rules make that safe.

2.1 Use the same encryption key everywhere

PAN and bank/UPI details are encrypted with `DATA_ENCRYPTION_KEY` before they are stored. Your local `.env` and the Render API must use the **identical** value. If they differ, records written by one cannot be read by the other.

> **Do not generate a new key**
> LAUNCH.md says to generate a new key for production. That applies only to a separate production project. Here, copy the value already in your `.env`. Store a copy in a password manager, not only on your laptop. If the key is lost, stored PAN and bank details cannot be recovered.

2.2 One-time clean-up before the first real partner

1. Take a backup (2.4).
2. Delete all test partners, leads, sales, commissions and payouts. Settings, commission plans, centres, legal text, templates and admin users are kept:

```
npm run reset-test-data -w @hoc/api -- --yes
```

3. In Supabase → Authentication → Users, delete the test partner login users (the script does not remove them), and in Storage → `partner-documents` delete the test uploads.
4. Supabase → Authentication → Sign In / Providers → Phone → **Test phone numbers: empty**. The only exception is the Play Store reviewer number (Section 3, Step 6), and it must have an expiry date.

> **After go-live, never run `reset-test-data` again**
> The script refuses to run only when `NODE_ENV=production`. Your local `.env` is development, so from your laptop it *will* wipe every real partner, lead and commission.

2.3 Working locally after go-live

- **Prefer not to run the local API** against the shared database. When it runs, it also runs the 10-minute jobs on real data. They are safe to repeat, but they can send duplicate push notifications.
- If you need to test, use **one dedicated test partner** named clearly, for example "TEST – do not pay". Never use a real customer's phone number in a test lead. Pilot metrics will include this partner, so keep its activity small.
- Do not change settings, commission rates or legal text "just to try something". The live app reads them immediately.
- **Migrations** (`npm run db:migrate`) apply directly to production. Take a backup first, and deploy the matching API version straight after.

2.4 Backups on the free plan

The Supabase free plan has no automatic daily backups. Take one weekly, before every migration, and before 2.2:

```
# needs the Postgres client tools: brew install libpq (use pg_dump 17 or newer)
pg_dump "$DATABASE_URL" -Fc -f "ho-connect-$(date +%F).dump"
```

Keep the files off your laptop, for example in a private company drive. Upgrade to Supabase Pro (~$25/mo, daily backups) before the first payout batch is paid.

2.5 Other free-tier limits to know

- Supabase pauses free projects after about a week with no activity. The API's 10-minute jobs keep the project active as long as the API is running.
- Before onboarding day, raise the SMS-per-hour limit under Authentication → Rate Limits.
- Free tier: 500 MB database and 1 GB storage. That is plenty for 25 partners. Check usage monthly under Settings → Usage.

3 Deployment, step by step

Do these in order. Each step ends with a check. Do not move on until it passes.

Step 0 · Put the code on GitHub

Render, Vercel and EAS all build from git, and this repository has no commits yet.

```
git add -A
git status                          # confirm no .env file is listed (.gitignore excludes them)
git commit -m "HO Connect MVP"
gh repo create <your-account>/ho-connect --private --source . --push
```

Check: the GitHub repository is **private** and contains no `.env` or `supabase/functions/.env`.

Step 1 · Verify the existing Supabase project (no new project)

```
npm run db:migrate                  # applies anything missing, including 0004_custom_qr_codes
supabase functions deploy send-sms --project-ref <ref>     # redeploy only if the function changed
supabase secrets set --env-file supabase/functions/.env --project-ref <ref>
```

- [ ] Authentication → Auth Hooks: the **Send SMS** hook points to `https://<ref>.supabase.co/functions/v1/send-sms`.
- [ ] Phone provider is on, OTP length 6, expiry 300 seconds.
- [ ] Storage: the `partner-documents` bucket exists and is **private**.
- [ ] At least one admin user exists (`npm run create-admin -w @hoc/api` if not).

Step 2 · Deploy the API on Render

1. render.com → New → **Web Service** → connect the GitHub repository.
2. Language: **Docker**. Dockerfile path `deploy/Dockerfile.api`, Docker build context `.` (the repository root).
3. Region: **Singapore** (the closest region to India). Instance type: **Free**. Instance count: **1**.
4. Health check path: `/health/ready`.
5. Environment variables: see the API table in Section 5. Do not set `PORT`; Render provides it and the API reads it. For now, set `CORS_ORIGINS` and `PUBLIC_WEB_URL` to `http://localhost:3000`. Step 4 fixes them.
6. Deploy. Note the URL, e.g. `https://ho-connect-api.onrender.com`.
7. **Keep it awake.** The free tier sleeps after 15 minutes without traffic, and the background jobs stop while it sleeps. At cron-job.org (free), create a job: `GET https://<api>/health` every **10 minutes**.

Check: `https://<api>/health/ready` returns OK, and the Render logs show no `background job failed` lines.

> **Free tier limits, accepted for the pilot**
>
> Even with the ping, a free instance can restart; the first request after a restart takes about 30–60 seconds. Every deploy also briefly takes the API offline. Before the first payout, move to Render Starter (~$7/mo): it never sleeps, deploys without downtime, and needs no ping. The free always-on alternative is an Oracle Cloud Always Free VM running the same Docker image, but you have to maintain that server yourself.

Step 3 · Deploy the dashboard on Vercel

1. vercel.com → Add New → Project → import the repository.
2. Root Directory: `apps/dashboard`. Framework preset: **Next.js**. Keep the default install and build commands, and keep "Include files outside the root directory" **on** (the dashboard imports `@hoc/shared`).
3. Environment variables: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` (the publishable key), `NEXT_PUBLIC_API_URL` = the Render URL from Step 2.
4. Deploy. Note the URL, e.g. `https://ho-connect.vercel.app`, or add your own domain such as `connect.hairoriginals.com`.

> **Two things to know about Vercel**
>
> - The `NEXT_PUBLIC_*` values are fixed when the dashboard is built. If the API address changes, redeploy the dashboard.
> - Vercel's free Hobby plan is for non-commercial use. For a company pilot, plan on Vercel Pro, or deploy `deploy/Dockerfile.dashboard` as a second Render service instead (it would need the keep-alive ping as well).

Step 4 · Connect the API to the dashboard

On Render, set `CORS_ORIGINS` and `PUBLIC_WEB_URL` to the dashboard address (no trailing slash). The deploy restarts automatically.

Check: log in at `https://<dashboard>/login`, open Partners and Leads, and open a partner's QR link `https://<dashboard>/r/<code>` on your phone.

Step 5 · Build the Android app with EAS

```
cd apps/mobile
npx eas-cli login
npx eas-cli init                    # copy the project id it prints

for kv in EXPO_PUBLIC_SUPABASE_URL=https://<ref>.supabase.co \
          EXPO_PUBLIC_SUPABASE_ANON_KEY=<publishable key> \
          EXPO_PUBLIC_API_URL=https://<api> \
          EXPO_PUBLIC_WEB_URL=https://<dashboard> \
          EXPO_PUBLIC_EAS_PROJECT_ID=<project id>; do
  npx eas-cli env:create --environment production --visibility plaintext \
    --name "${kv%%=*}" --value "${kv#*=}" --non-interactive
done

npx eas-cli build --platform android --profile preview      # APK: install on your own phone first
npx eas-cli build --platform android --profile production   # .aab for the Play Store
```

- [ ] **Push notifications:** in Firebase, add the Android app `com.hairoriginals.connect`, put `google-services.json` in `apps/mobile`, set `android.googleServicesFile` in `app.config.ts`, and upload the FCM key with `npx eas-cli credentials`. Then rebuild.

- [ ] Replace the placeholder icon in `apps/mobile/assets` with the final mark (same file names) before the production build.
- [ ] **Check:** on the preview APK, log in with a real number. The SMS arrives, a lead can be added, and a push notification arrives.

Step 6 · Google Play internal testing

1. Create the app in Play Console ($25 one-time): "HO Connect", English (India), free.
2. App content: privacy policy `https://<dashboard>/privacy`; account deletion `https://<dashboard>/delete-account`; Data safety, permissions (camera only), content rating and target audience as described in LAUNCH.md §5.
3. Reviewer access: an approved test partner, plus one Supabase test phone number with a fixed code **and an expiry date**. Enter both under App access. **Remove the number once the review passes.**
4. Store listing: descriptions, 512×512 icon, 1024×500 feature graphic, and screenshots taken from the preview build.
5. Testing → Internal testing → upload the `.aab` by hand the first time. After that, `npx eas-cli submit -p android --profile production` uploads new builds as drafts (requires a Google service-account key).
6. Add the pilot partners' Google email addresses as testers. Each tester gets an opt-in link.

Step 7 · Switch on production behaviour (cut-over day)

- [ ] Clean-up from Section 2.2 is done, and a backup was taken first.
- [ ] Render: `OTP_PROVIDER=nimbus` with all four `NIMBUS_*` values. The customer consent DLT template is set with `npm run message-template -w @hoc/api`.
- [ ] Final legal text, commission rates, TDS, minimum payout, hold period and referral benefit are entered.
- [ ] Run through TESTING.md once on the live system with the dedicated test partner: register, approve, add a lead, receive the customer code by SMS, record a sale, see the commission.

4 Monthly cost

Pilot start (free)

Service	Cost
Supabase Free	$0
Render Free + cron-job.org ping	$0
Vercel Hobby	$0
Expo EAS Free	$0
Google Play Console	$25 once
Domain (optional)	~$10 / year
Total	**$25 once + $0 / mo**

Before the first payout (recommended)

Service	Cost
Supabase Pro (daily backups)	~$25 / mo
Render Starter (always on)	~$7 / mo
Vercel Pro (commercial use)	~$20 / mo
Expo EAS Free	$0
Total	**~$52 / mo**

To spend less: run the dashboard as a second Render Starter service (~$7) instead of Vercel Pro. That brings the total to ~$39 / mo.

USD list prices, approximate. Check each provider's pricing page before you sign up.

5 Environment variables

same as .env means: copy the value from your current local `.env`. With one Supabase project, these must match.

API (Render)

Variable	Value	
`NODE_ENV`	`production`	required
`SUPABASE_URL`	`https://<ref>.supabase.co`	same as .env
`SUPABASE_SERVICE_ROLE_KEY`	Secret key (`sb_secret_…`)	same as .env server only
`SUPABASE_JWT_SECRET`	Leave unset unless your `.env` has it	same as .env
`DATABASE_URL`	Session pooler, port 5432	same as .env
`DATA_ENCRYPTION_KEY`	The existing key. **Never generate a new one.**	same as .env
`CORS_ORIGINS`	Dashboard URL (comma-separated if several)	required
`PUBLIC_WEB_URL`	Dashboard URL	required
`OTP_PROVIDER`	`console` until cut-over, then `nimbus`	required
`NIMBUS_USER_ID`, `NIMBUS_PASSWORD`, `NIMBUS_SENDER_ID` (`HAIROG`), `NIMBUS_ENTITY_ID`	Same Nimbus account as the try-on project	with nimbus
`PUSH_PROVIDER`	`expo`	required
`EXPO_ACCESS_TOKEN`	Only if enhanced push security is on	optional
`TRYON_WEB_URL`	Only if not `https://aitryon.hairoriginals.com`	optional
`PORT`, `HOST`	Do not set. Render provides `PORT`; `HOST` defaults to `0.0.0.0`.	

Dashboard (Vercel)

Variable	Value
`NEXT_PUBLIC_SUPABASE_URL`	same as .env
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Publishable key same as .env
`NEXT_PUBLIC_API_URL`	Render API URL

Mobile (EAS, production environment)

Variable	Value
`EXPO_PUBLIC_SUPABASE_URL`	same as .env
`EXPO_PUBLIC_SUPABASE_ANON_KEY`	Publishable key
`EXPO_PUBLIC_API_URL`	Render API URL
`EXPO_PUBLIC_WEB_URL`	Dashboard URL
`EXPO_PUBLIC_EAS_PROJECT_ID`	From `eas init`

Never put the secret key in the dashboard or the app

Only the **publishable** key goes into `NEXT_PUBLIC_*` and `EXPO_PUBLIC_*` values, because anyone can read them. The secret key and the encryption key belong only in Render and your local `.env`.

6 Go-live checklist

Business & legal

- [ ] Final partner terms and customer consent published (SQL in LAUNCH.md §1)
- [ ] Privacy policy and delete-account pages final, with a real support email
- [ ] DLT templates registered: customer consent and reminder (`customer_nudge`)
- [ ] Commission rates, TDS rate and threshold, minimum payout, hold period, referral benefit agreed with finance and entered
- [ ] Experience centre phone numbers and addresses correct
- [ ] Hindi texts read by a native speaker
- [ ] Final app icon in place

Shared Supabase project

- [ ] Backup taken and stored off the laptop
- [ ] Test data cleared (`reset-test-data` , auth users, storage)
- [ ] Test phone numbers empty (reviewer number has an expiry date)
- [ ] SMS rate limit raised for onboarding day
- [ ] Migrations up to date, including `0004_custom_qr_codes`
- [ ] `DATA_ENCRYPTION_KEY` and secret key in a password manager

Systems

- [ ] API on Render: `/health/ready` OK, exactly one instance
- [ ] Keep-alive ping running every 10 min
- [ ] Dashboard on Vercel: staff login works
- [ ] `CORS_ORIGINS` and `PUBLIC_WEB_URL` point to the dashboard
- [ ] `OTP_PROVIDER=nimbus` , and a real customer code arrives by SMS
- [ ] Push notification received on a real Android phone
- [ ] CAPTCHA (Cloudflare Turnstile) on the customer QR page, or a conscious decision to wait
- [ ] AI try-on app deployed with partner attribution; one try-on QR checked end to end into DigiCuro (TESTING.md TRY-6)
- [ ] Play internal testing release live; testers invited

Proof it works

- [ ] TESTING.md walked through once on the live system
- [ ] One real partner onboarded end to end
- [ ] One payout paid and its UTR entered

When to upgrade

Trigger	Action
Before the first payout batch is paid	Supabase Pro (backups) and Render Starter (always on)
Before publishing QR posters widely	CAPTCHA on `/r/<code>`
You need a safe place to test new features	Create a second, development-only Supabase project then. Nothing in the code has to change; only the `.env` values do.
Pilot extends beyond 25 partners or adds iOS	Revisit this plan: Apple developer account, iOS push, a staging environment.